

OOP Interview Smart Answers Guide

Encapsulation

Encapsulation ensures that object state is protected and can only be modified through controlled methods. Example: In a `BankAccount` class, `balance` should not be directly accessible. Instead, we expose `deposit()` and `withdraw()` methods. This ensures validation and maintains consistency. In real backend systems, this helps maintain invariants and prevents unexpected side effects.

Abstraction

Abstraction hides implementation details and exposes only essential behavior. Example: In a payment system, a `Payment` interface defines `pay()`. The client doesn't need to know whether it's UPI, Card, or NetBanking internally. This reduces coupling and makes the system flexible and extendable. Commonly used in layered architecture and service-based backend systems.

Inheritance

Inheritance allows one class to acquire properties and behaviors of another. Example: A base class `Vehicle` can have subclasses `Car` and `Bike` sharing common properties like `speed` and `fuelCapacity`. However, in scalable systems, composition is often preferred over inheritance to reduce tight coupling.

Polymorphism

Polymorphism allows the same interface to behave differently depending on implementation. Example: Define a `Payment` interface and implement it via `CreditCardPayment`, `UpiPayment`, etc. This enables adding new payment methods without modifying existing client code, aligning with the Open-Closed Principle. Commonly implemented using the Strategy Pattern in LLD.